Using the AIX Lab-Atlas skills

in Claude Design

Version	3
For	Designers working in Claude Design. Setup takes about 10 minutes, once per project.
Source	https://github.com/RPalis/AIXLab-Atlas. This is a team guide, not an official Anthropic document.
Licence	MIT. Free to use, with the licence notice.

Step 1. Install

Follow **Part B** of the Install click-by-click guide. It connects the repository, copies the 30 skills into your project and adds the rule file, in about 10 minutes. Repeat it in every new project.

Step 2. Invoke a skill

Type a slash, the skill name, and an optional mode or idea, then send. If you do not know where to start, send `/help`. It reads your project's files and recommends the next one to three skills, each tied to a file it found or did not find.

```
/help
/help atlas
/brief an app to share weekly meal plans with housemates
/audit craft
```

The slash is optional. "Run the audit skill in craft mode" does the same thing.

The commands

Phase	Commands (modes in brackets)
Before Discover	`/setup`, `/brief`
Discover	`/workshop` [plan, ingest], `/competitor-analysis` [discover, add, analyse], `/ux-benchmark`, `/interview`, `/interview-analysis`, `/persona` [card, day], `/journey` [map, blueprint, pain-points, solutions, experience], `/problem-statement`, `/metrics`
Decide	`/brainstorming`, `/information-architecture`, `/map` [objects, site, flow, audit, document], `/interrogate`, `/decisions`
Design	`/prototype`, `/styleguide` [default, forge], `/audit` [craft, heuristic, code, spec, a11y], `/refine`, `/storybook`, `/figma` [capture, import, sync]
Deliver	`/usability-test-plan`, `/package` [full, lite], `/showcase`, `/guide`, `/requirements`
Any time	`/wiki` [compile, note], `/skill-manager`
Router	`/help` [default, atlas]

What each skill needs

Skills that need a shell and Node.js (`/prototype`, `/storybook`, `/package`, `/showcase`, `/skill-manager`) say so and stop. Run those in Claude Code.

Audit

Audit finds problems and never edits your design. `craft` scores five dimensions 0 to 4 and ranks a task list P0 to P3. `heuristic` is a 12-point pass for when the audit must stand as evidence. `a11y` checks WCAG 2.1 and 2.2 AA. `code` checks tokens and structure. `spec` checks the build against the design. Run one mode at a time, fix with `/refine`, then audit again. Never merge craft and heuristic scores.

Plan

1. **Place the project.** Send `/help`. It reads which docs exist and recommends the next skill. An empty project gets `/setup`, then `/brief`.
2. **Follow the phases:** Discover, Decide, Design, Deliver. You can skip ahead and backfill.
3. **Plan before writing.** Ask each skill for its plan first, and wait for your yes before it creates anything.

Guardrails

- **No invented evidence.** No made-up people, quotes, numbers or competitors. Missing facts go on a line in `UNKNOWNS.md`, and Claude asks you.
- **Name the source** of every finding, and label any guess as a guess.
- **Stay in scope.** Audit never edits. Setup adds to existing files and never overwrites them.
- **Never claim what did not happen.** No "file written" unless it was.

Paste this before a skill run to enforce them:

```
Use only files in this project and the skill's own SKILL.md. Do not invent users, quotes,
numbers, competitors or screens. If something is missing, add it to UNKNOWNS.md and ask m
e.
Show me a plan first and wait for my yes before creating or changing files. Name the sourc
e
file for every finding. If you guess, say it is a guess.
```

Avoiding slop

- **Text:** no praise words with nothing behind them (seamless, powerful), no big words for plain ones (leverage, utilize), no vague sources ("studies show"), no chatbot closers. Text you supply stays verbatim.
- **Design:** no filler sections, placeholder copy or invented stats. No gradients, emoji or stock fonts unless your brand uses them. Claude cannot make photos and uses labelled placeholders. Small request, small change.

How to: design systems

A design system is built from sources you give Claude Design: code, documents or decks, and brand assets. **Foundations** means colour, type, spacing, radius and shadow in light and dark, with no components. Background: Claude Design help.

1. Connect a design system from Git

1. Open claude.ai/design. On Enterprise, an owner first turns on **Design systems** in **Organization settings > Artifacts**.
2. Open **Settings > Design systems** and start a new one, or send in chat: `Create a design system from the GitHub repo <owner>/<repo>, branch main.`
3. Click **Open**, then **Remix**, to review and tidy it. Make a test project and check colours, type and components.
4. Turn on **Published** so new projects use it. For a React component library, run `/design-sync` from Claude Code for a fuller import.

2. Create a design system (foundations only)

1. Gather your logo files, colour file, type spec or brand deck. Open **Settings > Design systems**, start a new one, name it and upload them.
2. Send: `Foundations only: colour, typography, spacing, radius and shadow, light and dark. No components. Use only my files and ask if something is missing.`
3. Click **Open**, then **Remix**, and check every colour and font against your brand guide. Make a test project, then turn on **Published**.

3. Connect a component library and style guide from Figma

1. In Figma, open the component library file and choose **File > Save local copy** to get a `.fig` file. Do the same for the style guide file if it is separate.
2. In **Settings > Design systems**, start a new one and name it. Look for the option to upload a `.fig` file, upload the library, and say yes when asked to import everything.
3. Add the style guide (`.fig`, PDF or deck) plus your brand fonts and logos. Fonts often do not carry over from Figma.
4. Click **Open**, then **Remix**, check colours, fonts, spacing and components, and fix them by chat. Make a test project, then turn on **Published**.

> **NOTE** A Figma import is less exact than a code import, and components are the weakest part. If your components also exist in Git, do how-to 1 as well.

Why the skills do not appear when you type "/"

Typing `/` in the Claude Design input box shows no list of these skills, and you cannot add them to it from inside a project. They are plain files in your project, not features the app has registered. They work anyway because `CLAUDE.md` tells the assistant to read `skills/<name>/SKILL.md` when a message starts with `/<name>`. Type the full command and send it. `/help atlas` was tested this way and ran. If a command does nothing, check that the rule file step in Part B was done in this project and that `skills/<name>/SKILL.md` exists.

If something goes wrong

What you see	What to try
A command does nothing	Repeat Part B of the Install guide in this project, then check `skills/<name>/SKILL.md` exists.
Anything else, or a skill writes before showing a plan	Type: stop, show me your plan first. Then call the AIX-Lab support channel.